

Lecture 1

Review of COMP 110

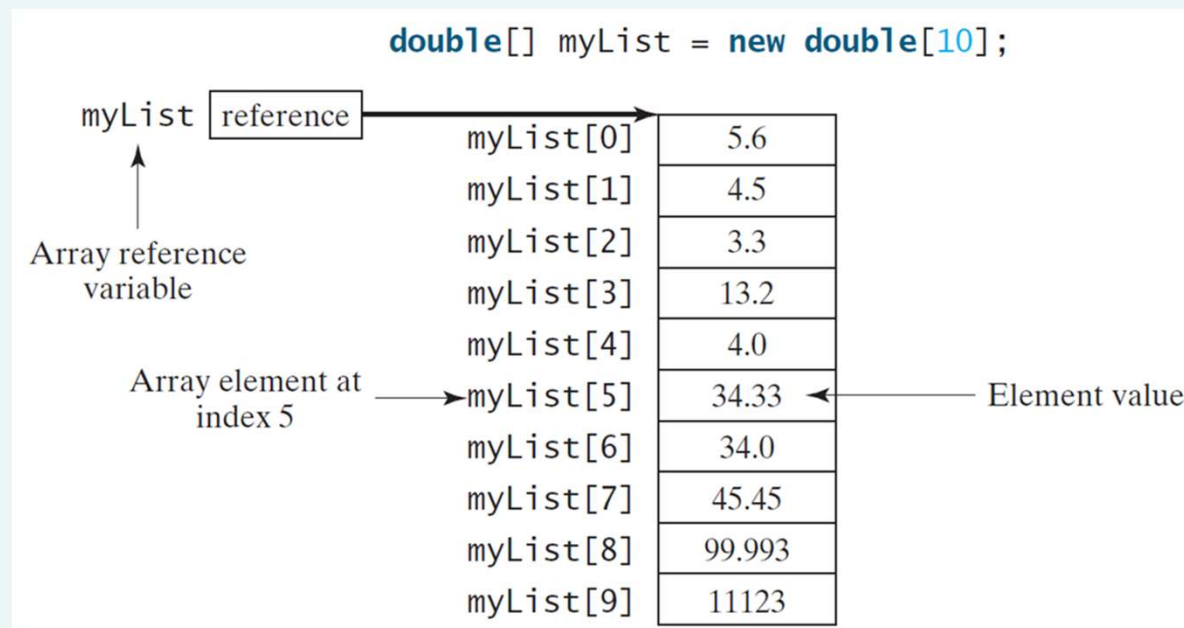
Single-Dimensional Arrays

Objectives

- To describe why arrays are necessary in programming
- To declare array reference variables and create arrays
- To obtain array size using **arrayRefVar.length** and know default values in an array
- To access array elements using indexes
- To declare, create, and initialize an array using an array initializer
- To program common array operations (displaying arrays, summing all elements, finding the minimum and maximum elements, random shuffling, and shifting elements)
- To simplify programming using the foreach loops

Introducing Arrays

Array is a data structure that represents a collection of the same types of data.



Array Initializers

- Declaring, creating, initializing in one step:

```
double[] myList = {1.9, 2.9, 3.4, 3.5};
```

This shorthand syntax must be in one statement.

Processing Arrays

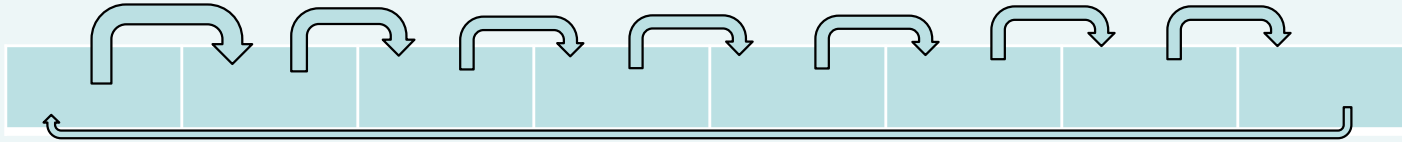
See the examples in the text.

1. (Initializing arrays with input values)
2. (Initializing arrays with random values)
3. (Printing arrays)
4. (Summing all elements)
5. (Finding the largest element)
6. (Finding the smallest index of the largest element)
7. (**Random shuffling**)
8. (**Shifting elements**) - left

Shifting Elements – shift left



Shifting Elements – shift right



Enhanced for Loop (for-each loop)

JDK 1.5 introduced a new for loop that enables you to traverse the complete array sequentially without using an index variable. For example, the following code displays all elements in the array `myList`:

```
for (double value: myList)  
    System.out.println(value) ;
```

In general, the syntax is

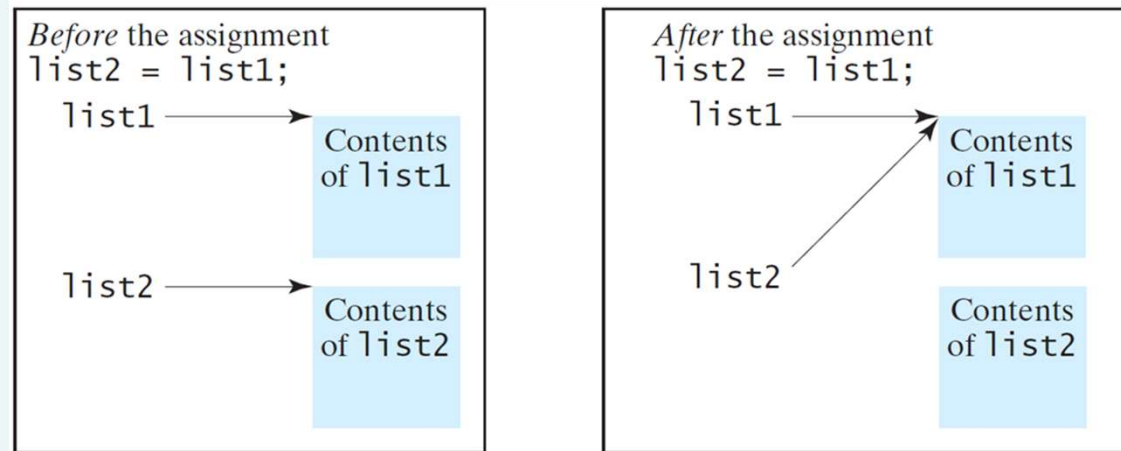
```
for (elementType value: arrayRefVar) {  
    // Process the value  
}
```

You still have to use an index variable if you wish to traverse the array in a different order or change the elements in the array.

Copying Arrays (1 of 2)

Often, in a program, you need to duplicate an array or a part of an array. In such cases you could attempt to use the assignment statement (=), as follows:

```
list2 = list1;
```



Copying Arrays (2 of 2)

Using a loop:

```
int[] sourceArray = {2, 3, 1, 5, 10};  
int[] targetArray = new  
    int[sourceArray.length];  
  
for (int i = 0; i < sourceArray.length;  
    i++)  
    targetArray[i] = sourceArray[i];
```

The arraycopy Utility

```
arraycopy(sourceArray, src_pos,  
          targetArray, tar_pos, length);
```

Example:

```
System.arraycopy(sourceArray, 0,  
                 targetArray, 0, sourceArray.length);
```

Passing Arrays to Methods

```
public static void printArray(int[] array) {  
    for (int i = 0; i < array.length; i++) {  
        System.out.print(array[i] + " ");  
    }  
}
```

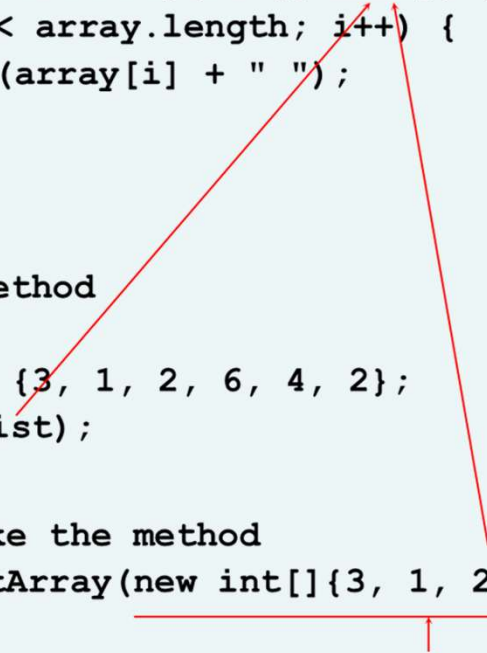
Invoke the method

```
int[] list = {3, 1, 2, 6, 4, 2};  
printArray(list);
```

Invoke the method

```
printArray(new int[]{3, 1, 2, 6, 4, 2});
```

Anonymous array



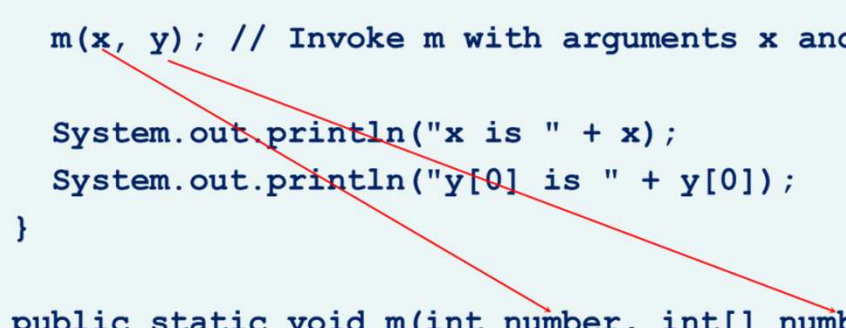
Pass by Value

Java uses **pass by value** to pass arguments to a method. There are important differences between passing a value of variables of primitive data types and passing arrays.

- For a parameter of a primitive type value, the actual value is passed. Changing the value of the local parameter inside the method does not affect the value of the variable outside the method.
- For a parameter of an array type, the value of the parameter contains a reference to an array; this reference is passed to the method. Any changes to the array that occur inside the method body will affect the original array that was passed as the argument.

Simple Example

```
public class Test {  
    public static void main(String[] args) {  
        int x = 1; // x represents an int value  
        int[] y = new int[10]; // y represents an array of int values  
  
        m(x, y); // Invoke m with arguments x and y  
  
        System.out.println("x is " + x);  
        System.out.println("y[0] is " + y[0]);  
    }  
  
    public static void m(int number, int[] numbers) {  
        number = 1001; // Assign a new value to number  
        numbers[0] = 5555; // Assign a new value to numbers[0]  
    }  
}
```

A diagram consisting of two red arrows. The first arrow originates from the 'x' parameter in the 'm(x, y)' call within the 'main' method and points to the 'number' parameter in the 'm' method signature. The second arrow originates from the 'y' parameter in the 'm(x, y)' call and points to the 'numbers' parameter in the 'm' method signature.

Searching Arrays

Searching is the process of looking for a specific element in an array; for example, discovering whether a certain score is included in a list of scores. Searching is a common task in computer programming. There are many algorithms and data structures devoted to searching. In this section, two commonly used approaches are discussed, **linear search** and **binary search**.

```
public class LinearSearch {  
    /** The method for finding a key in the list */  
    public static int linearSearch(int[] list, int key) {  
        for (int i = 0; i < list.length; i++)  
            if (key == list[i])  
                return i;  
        return -1;  
    }  
}
```

list [0] [1] [2] ...
 □ □ □ □ □ □ □
key Compare key with list[i] for i = 0, 1, ...

Binary Search (1 of 6)

For binary search to work, the elements in the array must already be ordered. Without loss of generality, assume that the array is in ascending order.

e.g., 2 4 7 10 11 45 50 59 60 66 69 70 79

The binary search first compares the key with the element in the middle of the array.

Binary Search (2 of 6)

Consider the following three cases:

- If the key is less than the middle element, you only need to search the key in the first half of the array.
- If the key is equal to the middle element, the search ends with a match.
- If the key is greater than the middle element, you only need to search the key in the second half of the array.

Binary Search (1 of 6)

For binary search to work, the elements in the array must already be ordered. Without loss of generality, assume that the array is in ascending order.

e.g., 2 4 7 10 11 45 50 59 60 66 69 70 79

The binary search first compares the key with the element in the middle of the array.

Binary Search (2 of 6)

Consider the following three cases:

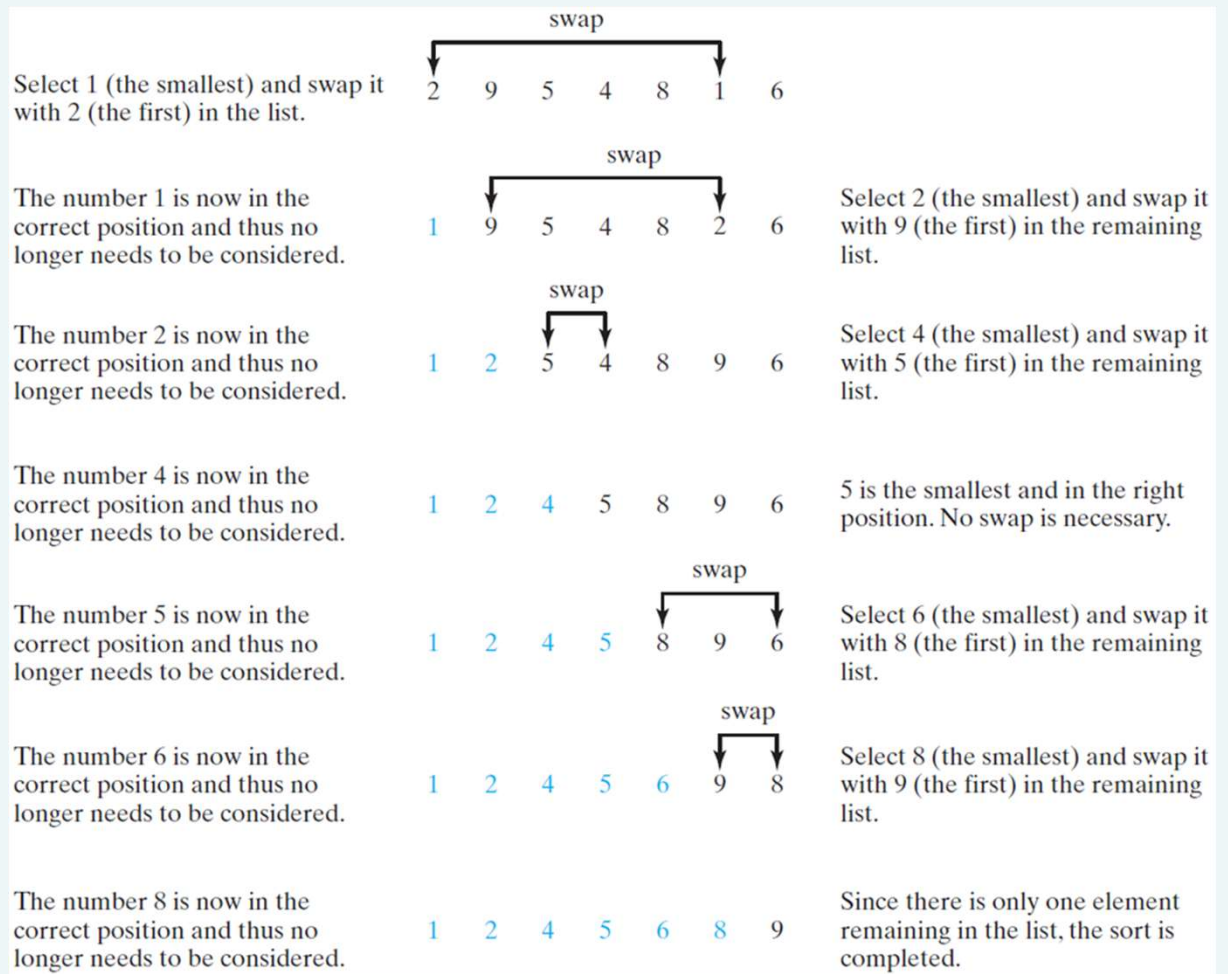
- If the key is less than the middle element, you only need to search the key in the first half of the array.
- If the key is equal to the middle element, the search ends with a match.
- If the key is greater than the middle element, you only need to search the key in the second half of the array.

Sorting Arrays

Sorting, like searching, is also a common task in computer programming. Many different algorithms have been developed for sorting. This section introduces a simple, intuitive sorting algorithm: **selection sort**.

Selection Sort

Selection sort finds the smallest number in the list and places it first. It then finds the smallest number remaining and places it second, and so on until the list contains only a single number.



Wrap it in a Method

```
/** The method for sorting the numbers */
```

```
public static void selectionSort(double[] list) {  
    for (int i = 0; i < list.length; i++) {  
        // Find the minimum in the list[i..list.length-1]  
        double currentMin = list[i];  
        int currentMinIndex = i;  
        for (int j = i + 1; j < list.length; j++) {  
            if (currentMin > list[j]) {  
                currentMin = list[j];  
                currentMinIndex = j;  
            }  
        }  
  
        // Swap list[i] with list[currentMinIndex] if necessary;  
        if (currentMinIndex != i) {  
            list[currentMinIndex] = list[i];  
            list[i] = currentMin;  
        }  
    }  
}
```

Invoke it

selectionSort(yourList)

The Arrays.sort Method

Since sorting is frequently used in programming, Java provides several overloaded sort methods for sorting an array of int, double, char, short, long, and float in the `java.util.Arrays` class. For example, the following code sorts an array of numbers and an array of characters.

```
double[] numbers = {6.0, 4.4, 1.9, 2.9, 3.4, 3.5};
```

```
java.util.Arrays.sort(numbers);
```

```
char[] chars = {'a', 'A', '4', 'F', 'D', 'P'};
```

```
java.util.Arrays.sort(chars);
```

Java 8 now provides `Arrays.parallelSort(list)` that utilizes the multicore for fast sorting.

The `Arrays.toString(list)` Method

The `Arrays.toString(list)` method can be used to return a string representation for the list.

Multidimensional Arrays

Motivations (1 of 2)

Thus far, you have used one-dimensional arrays to model linear collections of elements. You can use a two-dimensional array to represent a matrix or a table. For example, the following table that describes the distances between the cities can be represented using a two-dimensional array.

Distance Table (in miles)

	Chicago	Boston	New York	Atlanta	Miami	Dallas	Houston
Chicago	0	983	787	714	1375	967	1087
Boston	983	0	214	1102	1763	1723	1842
New York	787	214	0	888	1549	1548	1627
Atlanta	714	1102	888	0	661	781	810
Miami	1375	1763	1549	661	0	1426	1187
Dallas	967	1723	1548	781	1426	0	239
Houston	1087	1842	1627	810	1187	239	0

Objectives

- To give examples of representing data using two-dimensional arrays.
- To declare variables for two-dimensional arrays, create arrays, and access array elements in a two-dimensional array using row and column indexes.
- To program common operations for two-dimensional arrays (displaying arrays, summing all elements, finding the minimum and maximum elements, and random shuffling).
- To pass two-dimensional arrays to methods.

Declare/Create Two-dimensional Arrays

```
// Declare array ref var
dataType[][] refVar;
// Create array and assign its reference to
variable
refVar = new dataType[10][10];
// Combine declaration and creation in one
statement
dataType[][] refVar = new dataType[10][10];
// Alternative syntax
dataType refVar[][] = new dataType[10][10];
```

Motivations (2 of 2)

```
double[][] distances = {  
    {0, 983, 787, 714, 1375, 967, 1087},  
    {983, 0, 214, 1102, 1763, 1723, 1842},  
    {787, 214, 0, 888, 1549, 1548, 1627},  
    {714, 1102, 888, 0, 661, 781, 810},  
    {1375, 1763, 1549, 661, 0, 1426, 1187},  
    {967, 1723, 1548, 781, 1426, 0, 239},  
    {1087, 1842, 1627, 810, 1187, 239, 0},  
};
```

Declaring Variables of Two-dimensional Arrays and Creating Two-dimensional Arrays

```
int[][] matrix = new int[10][10];
```

or

```
int matrix[][] = new int[10][10];
```

```
matrix[0][0] = 3;
```

```
for (int i = 0; i < matrix.length; i++)
```

```
    for (int j = 0; j < matrix[i].length; j++)
```

```
        matrix[i][j] = (int) (Math.random() *  
        1000);
```

```
double[][] x;
```

Two-dimensional Array Illustration

	[0]	[1]	[2]	[3]	[4]
[0]	0	0	0	0	0
[1]	0	0	0	0	0
[2]	0	0	0	0	0
[3]	0	0	0	0	0
[4]	0	0	0	0	0

`matrix = new int[5][5];`

(a)

	[0]	[1]	[2]	[3]	[4]
[0]	0	0	0	0	0
[1]	0	0	0	0	0
[2]	0	7	0	0	0
[3]	0	0	0	0	0
[4]	0	0	0	0	0

`matrix[2][1] = 7;`

(b)

	[0]	[1]	[2]
[0]	1	2	3
[1]	4	5	6
[2]	7	8	9
[3]	10	11	12

```
int[][] array = {  
    {1, 2, 3},  
    {4, 5, 6},  
    {7, 8, 9},  
    {10, 11, 12}  
};
```

(c)

`matrix.length?` 5
`matrix[0].length?` 5

`array.length?` 4
`array[0].length?` 3

Declaring, Creating, and Initializing Using Shorthand Notations

You can also use an array initializer to declare, create and initialize a two-dimensional array. For example,

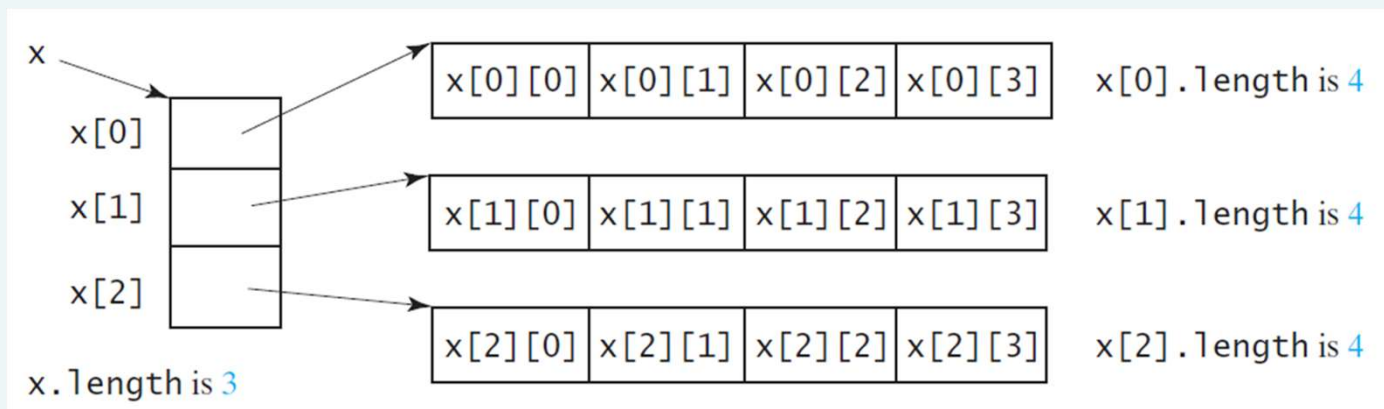
```
int[][] array = {  
    {1, 2, 3},  
    {4, 5, 6},  
    {7, 8, 9},  
    {10, 11, 12}  
};
```

Same as

```
int[][] array = new int[4][3];  
array[0][0] = 1; array[0][1] = 2; array[0][2] = 3;  
array[1][0] = 4; array[1][1] = 5; array[1][2] = 6;  
array[2][0] = 7; array[2][1] = 8; array[2][2] = 9;  
array[3][0] = 10; array[3][1] = 11; array[3][2] = 12;
```

Lengths of Two-dimensional Arrays (1 of 2)

```
int[][] x = new int[3][4];
```



Lengths of Two-dimensional Arrays (2 of 2)

```
int[][] array = {  
    {1, 2, 3},  
    {4, 5, 6},  
    {7, 8, 9},  
    {10, 11, 12}  
};
```

```
array.length  
array[0].length  
array[1].length  
array[2].length  
array[3].length
```

```
array[4].length
```

```
ArrayIndexOutOfBoundsException
```

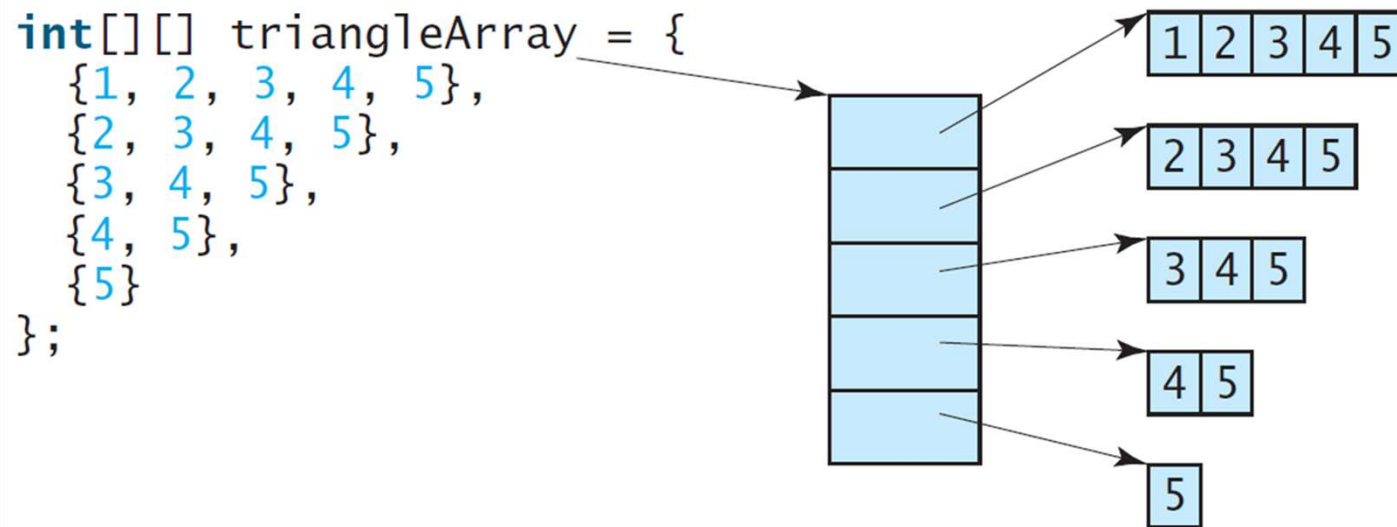
Ragged Arrays (1 of 2)

Each row in a two-dimensional array is itself an array. So, the rows can have different lengths. Such an array is known as a **ragged array**. For example,

```
int[][] matrix = {  
    {1, 2, 3, 4, 5},  
    {2, 3, 4, 5},  
    {3, 4, 5},  
    {4, 5},  
    {5}  
};
```

```
matrix.length is 5  
matrix[0].length is 5  
matrix[1].length is 4  
matrix[2].length is 3  
matrix[3].length is 2  
matrix[4].length is 1
```

Ragged Arrays (2 of 2)



Processing Two-Dimensional Arrays

See the examples in the text.

1. (Initializing arrays with input values)
2. (Printing arrays)
3. (Summing all elements)
4. (Summing all elements by column)
5. (Summing all elements by row)
6. (**Random shuffling**)
7. (Which row has the largest sum)
8. (Finding the smallest index of the largest element)

Passing Two-Dimensional Arrays to Methods

PassTwoDimensionalArray